



Universidad Distrital Francisco José de Caldas
Faculty of Engineering - Systems Engineering

Technical Report: Design, Simulation, and Validation of a Real-Time Food Surplus Redistribution Platform

Luna Alejandra Sandoval Rodríguez | Project Lead
Cristian Camilo Bonilla Lizarazo | Frontend Developer
Nicolás Rodríguez Granados | Backend Developer
Juan Sebastián Bravo Rojas | Database Specialist

Supervisor: Eng. Carlos Andrés Sierra

June 3, 2026

Abstract

In Colombia, approximately 9.7 million tons of food are wasted annually, exacerbating environmental damage, food insecurity, and social inequality — particularly within the university context of Bogotá. This technical report presents the complete systems analysis, architectural design, simulation validation, and functional prototype of a mobile platform for the real-time redistribution of surplus food suitable for human consumption. The platform connects university cafeterias and local restaurants with students, community members, and verified charitable organizations within a strictly enforced 3 km walking-access radius around the Faculty of Engineering at the Universidad Distrital Francisco José de Caldas.

The solution implements a four-layer architecture (external actors, mobile application, services and intelligence layer, and data and analytics layer), a priority-queue-based matching algorithm with PostGIS geospatial enforcement, staged proximity-tier notifications, and same-day operational constraints. The system was validated through two complementary simulation paradigms — a Discrete-Event Simulation (DES, Python SimPy 4.1) and an Agent-Based Model (ABM, Python Mesa 2.3.0) — executed across five experimental scenarios with 30 independent replications each ($n = 300$ total runs). Results confirm a Surplus Recovery Rate of 96–99%, a Charity Assignment Share consistently above 20%, and identify no-show behavior as the dominant system risk, with a critical threshold of 7% (DES) and 3–4% (ABM) for maintaining Pickup Completion Rate above 85%.

The report documents the full workshop-to-workshop evolution: Workshop 1 established the sociotechnical system analysis; Workshop 2 produced the architectural design and 17 traceable requirements; Workshop 3 refined risk management and quality assurance; and Workshop 4 validated system behavior through simulation. The functional prototype was implemented using React Native, NestJS, PostgreSQL/PostGIS, and Redis, and is containerized for reproducibility via Docker Compose.

Keywords: food waste reduction, surplus redistribution, geolocation, matching algorithm, mobile application, circular economy, simulation, agent-based model, discrete-event simulation, Bogotá, systems engineering

Contents

1	Introduction	1
2	Literature Review	2
3	Objectives	3
3.1	General Objective	3
3.2	Specific Objectives	3
4	Scope	4
4.1	Within System Scope	4
4.2	Outside System Scope	4
5	Assumptions	6
6	Limitations	7
7	Methodology	8
7.1	Workshop 1 — Sociotechnical Systems Analysis	8
7.2	Workshop 2 — System Architecture and Design	8
7.3	Workshop 3 — Risk Management and Quality Assurance	8
7.4	Workshop 4 — Computational Simulation and Validation	9
8	System Analysis (Workshop 1)	10
8.1	Problem Characterization	10
8.2	System Boundaries and IPO Model	10
8.3	Critical Sensitivity Factors	10
9	System Design (Workshop 2)	12
9.1	Four-Layer Architecture	12
9.2	Matching Algorithm Design	12
9.3	Technology Stack	13
9.4	Design Requirements	13
10	Risk Management (Workshop 3)	15
10.1	Quality Assurance Framework	16
11	Simulation and Validation (Workshop 4)	17
11.1	Discrete-Event Simulation (DES)	17
11.2	Agent-Based Model (ABM)	18
11.3	Validation Results	18

<i>CONTENTS</i>	iii
11.4 System-Level Interpretation	19
12 Implementation and Results	20
12.1 Functional Prototype	20
12.2 Key Performance Indicators	20
12.3 Risk Mitigation in Implementation	21
13 Discussion	22
14 Conclusions and Future Work	23
References	24
Appendices	25
A Simulation Parameters	25
B Traceability Matrix (Requirements → Workshop 1 Findings)	26

Glossary

Table 1: Glossary of Terms

Term	Definition
API	Application Programming Interface — a set of protocols enabling software communication. RESTful APIs connect the mobile client to back-end services.
ABM	Agent-Based Model — a simulation paradigm representing autonomous actors with individual decision rules; used in Workshop 4 (Mesa 2.3.0).
Bull Queue	Redis-based message queue for Node.js that serializes matching engine requests to prevent race conditions.
Circular Economy	Economic model aimed at eliminating waste through continuous resource reuse; foundational principle of the platform.
CO ₂ -eq	CO ₂ -equivalent — standardized unit for greenhouse gas emissions; the platform uses the IPCC factor of 2.5 kg CO ₂ per kilogram of food redistributed.
DES	Discrete-Event Simulation — a simulation paradigm that advances only on scheduled events; used in Workshop 4 (SimPy 4.1).
Docker Compose	Multi-container orchestration tool enabling one-command local deployment of PostgreSQL, Redis, and NestJS containers.
Expo	React Native framework providing on-device testing, GPS access, camera integration, and push notification delivery.
FCM	Firebase Cloud Messaging — Google’s cross-platform messaging service for real-time push notifications.
Geolocation	Identification of a real-world geographic position; enforces the 3 km radius constraint through PostGIS ST_DWithin queries.
IPO	Input–Process–Output — structured systems modeling framework mapping inputs through processes to measurable outputs.
Matching Algorithm	Weighted priority function composed of 50% proximity, 30% fairness, and 20% charity priority. Executed as a serialized Bull queue job.
NestJS	Node.js backend framework organizing platform services into four decoupled modules.
PostGIS	Spatial extension for PostgreSQL enabling ST_DWithin radius queries for 3 km boundary enforcement.
Prisma	ORM for Node.js providing compile-time type safety and predictable database migration workflows.
Priority Queue	Data structure that processes higher-priority elements first; favors verified charities for offers ≥ 10 kg.
React Native	Cross-platform mobile framework enabling Android and iOS deployment from a single JavaScript codebase.
Redis	In-memory data store used for Bull queue management, JWT caching, and peak-concurrency load reduction.
Redistribution Efficiency	Percentage of published listings successfully claimed within the allowed pickup window.
REQ	Requirement identifier — one of the 17 measurable design requirements defined in Workshop 2, each traceable to Workshop 1 findings.
Surplus Food	Food prepared but not consumed by the original provider, remaining safe for human consumption and eligible for redistribution.
Zustand	Lightweight React Native state-management library that minimizes boilerplate while managing UI state.

Chapter 1

Introduction

Food waste represents one of the most critical environmental, economic, and social challenges in Colombia and Latin America. According to the UNEP Food Waste Index Report 2021, Colombia generates approximately 9.7 million tons of food lost or wasted annually — an amount sufficient to feed the entire population of Bogotá for a full year. In Bogotá, university cafeterias and local restaurants produce significant daily surpluses of food suitable for human consumption; however, the lack of rapid coordination mechanisms and the short shelf life of perishable items result in a large proportion of this surplus ending up in landfills. The problem is particularly acute in the university environment surrounding the Faculty of Engineering at Universidad Distrital Francisco José de Caldas (Carrera 8 #40-62, Bogotá). Daily operations generate recurrent surpluses, yet no efficient digital platform exists to enable immediate, geospatially constrained redistribution. Existing solutions such as OLIO and Eat-Cloud have demonstrated the potential of mobile applications to reduce food waste, but none combines strict geospatial enforcement, same-day expiry, tiered priority for verified charitable organizations, and a university-focused deployment model.

The Food Waste Reduction Platform was designed to address this gap. Developed through a rigorous four-workshop iterative process, the platform operates as a mobile coordination layer connecting donors (cafeterias, restaurants) with recipients (students, community members, verified foundations) within a 3 km radius. The system enforces same-day redistribution (publication 2–4 hours before closing; pickup within 1–3 hours) and focuses exclusively on digital coordination, leaving physical logistics, food safety verification, and financial transactions outside its scope.

This technical report integrates the complete project documentation: the sociotechnical systems analysis (Workshop 1), the architectural design and 17 measurable requirements (Workshop 2), the risk management and quality assurance framework (Workshop 3), and the computational simulation validation (Workshop 4). By combining quantitative simulation data ($n = 300$ runs), architectural diagrams, performance metrics, and the functional prototype, the report demonstrates the technical viability and expected socio-environmental impact of the platform.

Chapter 2

Literature Review

Mobile applications for food surplus redistribution have emerged as a viable technological response to the global food waste crisis. OLIO, launched in the United Kingdom, enables peer-to-peer sharing of surplus food through geolocation-based listings. While its community-driven model demonstrates redistribution potential at scale, it lacks organizational priority mechanisms and same-day operational constraints, limiting its applicability in time-sensitive university contexts.

In Colombia, EatCloud has pioneered data-driven food waste reduction by connecting food businesses with social organizations through a digital marketplace. Although effective at the commercial level, it is not designed for the geographically constrained, high-frequency donation cycles characteristic of university cafeteria environments. Similarly, the 'Yo Contribuyo, No Desperdicio' initiative promoted by the Alcaldía de Bogotá in October 2024 highlights growing institutional awareness of the problem but focuses on awareness campaigns and inter-sector coordination rather than operational digital infrastructure.

From a data standpoint, the UNEP Food Waste Index Report 2021 provides the quantitative foundation for this project. Analysis of the dataset across 214 countries reveals that Colombia wastes approximately 114 kg per capita per year, below the global average (126.8 kg) but reaching an estimated 5.72 million tons annually. Notably, 61% of this waste occurs at the household level — a segment difficult to address directly through a platform-based approach. However, university cafeterias and commercial food services represent a structurally more tractable intervention point, as surplus generation is recurrent, predictable, and concentrated within defined geographic and temporal windows.

From a technical standpoint, priority-based matching algorithms have been validated in resource allocation systems such as ride-sharing and emergency dispatch platforms. Adapting these mechanisms to food redistribution introduces fairness and efficiency requirements that standard first-come-first-served models cannot satisfy, particularly when verified charitable organizations must receive precedence. Geospatial databases such as PostgreSQL with the PostGIS extension enable efficient radius-based queries critical for enforcing geographic constraints, a pattern well validated in location-aware mobile services.

The literature thus reveals a gap: no existing platform combines strict geospatial constraints (3 km radius), same-day expiry rules, tiered priority for verified charitable organizations, race condition mitigation, and a university-focused deployment model. The present work addresses this gap directly, and validates its design through computational simulation before prototype implementation.

Chapter 3

Objectives

3.1 General Objective

To design, simulate, and validate a mobile platform for the real-time redistribution of surplus food generated by university cafeterias and partner restaurants within a 3 km radius of the Faculty of Engineering at Universidad Distrital Francisco José de Caldas, prioritizing verified charitable organizations and enforcing same-day operational constraints.

3.2 Specific Objectives

- To conduct a comprehensive sociotechnical systems analysis of the food waste problem in the university environment, identifying key stakeholders, system boundaries, sensitivity factors, emergent behaviors, and chaotic dynamics (Workshop 1).
- To design a four-layer system architecture integrating a React Native frontend, a NestJS backend, a PostGIS-based geolocation service, a priority-queue matching algorithm, staggered push notifications via FCM, and a centralized analytics layer (Workshop 2).
- To define 17 measurable design requirements traceable to Workshop 1 findings, covering functional constraints, reliability, performance, scalability, security, availability, usability, and maintainability dimensions (Workshop 2).
- To establish a structured risk management and quality assurance framework addressing the 10 critical sensitivity factors, emergent behaviors, and chaotic dynamics identified in the systems analysis (Workshop 3).
- To validate system behavior through two complementary computational simulation paradigms — Discrete-Event Simulation (DES) and Agent-Based Model (ABM) — executed across five experimental scenarios with 30 independent replications each (Workshop 4).
- To implement and document a functional prototype serving as an empirical foundation for real-world pilot deployment.

Chapter 4

Scope

The platform is designed to facilitate the redistribution of surplus food generated by university cafeterias and partner restaurants to students, community members, and charitable organizations within a strictly enforced 3 km radius centered on the Faculty of Engineering, Universidad Distrital Francisco José de Caldas (Carrera 8 #40-62, Bogotá). The system focuses on digital coordination through a mobile application, enabling stakeholders to publish, discover, and reserve surplus food items in real time.

4.1 Within System Scope

- Registration and management of all user types: students, community members, charities, cafeterias, and partner restaurants.
- Geolocation enforcement: all matching restricted to a 3 km radius via PostGIS ST_DWithin queries.
- Matching algorithm: priority-queue-based assignment with composite scoring (proximity, reliability, charity priority).
- Staged notifications: proximity-tier push notification delivery (500 m → 1 km → 3 km) via FCM.
- Reassignment pipeline: automatic up to 3 attempts per surplus, with compressed timeouts.
- Analytics and KPI dashboards: real-time monitoring for administrators and individualized impact reports for donors.
- User reliability scoring: dynamic score updates penalizing no-shows to reduce behavioral risk.

4.2 Outside System Scope

- Physical transportation or delivery logistics of food items.
- Food safety verification, quality inspection, or regulatory compliance certification.
- Payment processing or any form of financial transaction between parties.
- Multi-day listing management: all items expire at end of daily operating window.

- Cold-chain logistics or temperature-controlled redistribution.
- Surplus generated outside the 3 km operational radius.

Chapter 5

Assumptions

1. All participating cafeterias and restaurants are located within a 3 km walking-access radius of the Faculty of Engineering (Carrera 8 #40-62, Bogotá).
2. Donors publish surplus listings between 2 and 4 hours before their daily closing time; all pickups are completed within 1 to 3 hours of publication, within the 7:00 a.m.–9:00 p.m. operating window.
3. Users have access to a smartphone with an active internet connection and functional GPS capability within the operational area.
4. Verified charitable organizations and student groups complete a one-time registration and verification process before accessing priority-tier notifications.
5. The geolocation service (Google Maps API, with OpenStreetMap as fallback) provides coordinate accuracy within ± 50 m, sufficient for the 3 km radius enforcement.
6. The matching algorithm operates within a serialized Bull queue that prevents race conditions during concurrent surplus claim attempts.
7. Food quality and safety verification remain the responsibility of donors and recipients; the platform provides digital coordination only.
8. Push notification delivery via FCM reaches active registered users within 5 seconds for at least 95% of dispatch events (REQ-12).

Chapter 6

Limitations

1. **Geographic constraint:** The platform is strictly limited to a 3 km radius. Surplus food generated outside this boundary is excluded regardless of proximity to potential recipients.
2. **Same-day expiry:** All listings expire at the end of the daily operating window. Unclaimed items after three reassignment attempts are considered lost within the platform.
3. **No physical logistics:** The system does not manage transportation or packaging. Redistribution depends on recipients being physically able to collect food from the donor location.
4. **No food safety enforcement:** The platform cannot verify food condition, preparation standards, or allergen content. Compliance rests with donors and recipients.
5. **Geolocation dependency:** The matching algorithm requires valid GPS data. Manual address input fallback is implemented (available within 5 s, REQ-04), but prolonged outages reduce operability.
6. **Concurrency ceiling:** The current architecture is designed for up to 200 concurrent users (REQ-09). Demand beyond this threshold may degrade matching response times.
7. **No-show threshold:** Both simulations establish that maintaining Pickup Completion Rate $\geq 85\%$ requires system-level no-show rates below 7% (DES) or 3–4% (ABM). Exceeding these thresholds requires behavioral interventions outside the platform's technical scope.
8. **Prototype scope:** Implementation covers local Docker Compose deployment only. No remote cloud deployment is included in the current prototype phase.

Chapter 7

Methodology

The project followed an iterative, workshop-based methodology structured across four stages, each building upon the outputs of the previous one. This progression reflects a deliberate sociotechnical design philosophy: understanding the complete system first, then deriving architectural decisions from that understanding — rather than beginning with software design and working backward.

7.1 Workshop 1 — Sociotechnical Systems Analysis

A comprehensive analysis of the food waste problem was conducted using two complementary approaches: (1) collection from reliable secondary sources, including the UNEP Food Waste Index Report 2021, governmental publications from Minambiente and DNP, and Alcaldía de Bogotá initiatives; and (2) primary data metrics to be generated by the platform prototype through logs of postings, matches, claims, and confirmed pickups.

The analysis established: system boundaries and operational constraints (3 km radius, same-day window); a four-subsystem decomposition (User Interaction, Surplus Management, Assignment and Coordination, Monitoring and Analysis); an input-process-output (IPO) model; a stakeholder taxonomy across users, operators, administrators, and external actors; and a sensitivity analysis identifying 10 critical factors including high-demand events, concurrency spikes, no-show cascades, demand amplification through notifications, and informal coordination networks.

7.2 Workshop 2 — System Architecture and Design

Based on Workshop 1 findings, 17 measurable design requirements were derived and classified by category. A four-layer system architecture was designed: external actors layer, application layer (React Native + Expo), service and intelligence layer (NestJS, PostGIS, Bull queue, FCM), and data and analytics layer (PostgreSQL, custom KPI views). Ten critical risks received explicit mitigation patterns. Implementation was organized in three sequential phases aligned with the 2026-I academic semester.

7.3 Workshop 3 — Risk Management and Quality Assurance

Architectural improvements were introduced: asynchronous messaging via a message broker (RabbitMQ/Redis Streams) decoupled the Matching Algorithm from the Notification Service; service abstraction encapsulated external geolocation providers behind a clean interface. A

four-layer quality assurance framework was established (unit → integration → acceptance → pilot), with quantitative acceptance criteria tied directly to the 17 design requirements. Continuous improvement was embedded through daily stand-ups, post-experiment retrospectives, and closed-loop KPI monitoring.

7.4 Workshop 4 — Computational Simulation and Validation

Two complementary simulation paradigms were developed and executed across five experimental scenarios with 30 independent replications each ($n = 300$ total): a Discrete-Event Simulation (DES) implemented in Python SimPy 4.1, and a Behavior-Oriented Agent-Based Model (ABM) implemented in Python Mesa 2.3.0. The DES evaluated operational flow, matching latency, and infrastructure throughput. The ABM evaluated individual agent decisions, reliability score dynamics, and emergent collective behaviors. All replications used deterministic seeds (0–29 per scenario) for full reproducibility.

Chapter 8

System Analysis (Workshop 1)

8.1 Problem Characterization

The food waste problem in the university environment is fundamentally a coordination failure, not a supply or demand deficit. Cafeterias and restaurants generate predictable daily surpluses within defined temporal windows, and a population of students and community members within walkable distance would consume that surplus if they knew about it in time. The barrier is the absence of a real-time coordination mechanism that matches supply to demand before the freshness window closes.

This framing distinguishes the present work from general food waste reduction platforms: the intervention target is the coordination gap between donors and recipients within a geographically constrained, time-pressured environment — not the broader food system. The platform's design follows directly from this diagnosis.

8.2 System Boundaries and IPO Model

The system is delimited by functional, spatial, temporal, and technological boundaries. The spatial boundary is a 3 km radius centered on the Faculty of Engineering (Carrera 8 #40-62, Bogotá), ensuring pedestrian-accessible collection within the freshness window. The temporal boundary is a same-day operating window (7:00 a.m.–9:00 p.m.), with surplus publication 2–4 hours before closing and pickup within 1–3 hours of publication.

The IPO model structures the system's value chain: Inputs include surplus food descriptions, GPS coordinates, user profiles, and real-time collection requests. Processes encompass surplus publication, automatic matching, notification dispatch, collection tracking, and KPI aggregation. Outputs are redistributed food, impact metrics (kg recovered, CO avoided), and transaction confirmations.

8.3 Critical Sensitivity Factors

The sensitivity analysis identified 10 critical factors across three categories: **Critical input sensitivities:** High-demand surplus events generating race conditions; large volumes of simultaneous users degrading response times; geolocation service failures halting the matching pipeline; and confirmed matches with no pickup creating reassignment cascades. **Emergent behaviors:** Informal priority networks among users bypassing the algorithm; supplier disengagement under low demand creating negative reinforcement cycles; and inequitable geographic distribution due to proximity bias in the matching algorithm. **Chaotic system dynamics:**

Nonlinear cascade from a single no-show exhausting reassignment cycles before surplus expiry; demand amplification through simultaneous notifications creating contention spikes; and a negative feedback loop between poor performance metrics and supplier withdrawal.

These 10 factors directly informed every architectural decision in Workshop 2 and every simulation scenario in Workshop 4; the traceability between system diagnosis and design choices is the primary methodological contribution of this project.

Chapter 9

System Design (Workshop 2)

9.1 Four-Layer Architecture

The architecture is organized into four layers with strict separation of concerns:

- **External Actors Layer:** Restaurants/cafeterias (donors) and students/volunteers/charities (recipients) interact with the system exclusively through the mobile application.
- **Application Layer:** React Native + Expo mobile application and User Management Interface (Backend for Frontend). Provides GPS access, camera-based collection confirmation, and push notification reception.
- **Service and Intelligence Layer:** NestJS backend organized into four modules — SurplusModule (registration and lifecycle), MatchingModule (priority-queue engine), NotificationModule (FCM dispatch), and AnalyticsModule (KPI aggregation). The Geolocation Service (PostGIS + external Maps APIs) and Assignment Coordination component operate here. A Message Broker (RabbitMQ/Redis Streams, introduced in Workshop 3) decouples the Matching Engine from the Notification Service asynchronously.
- **Data and Analytics Layer:** PostgreSQL with PostGIS extension for spatial queries; Redis for caching and queue management; Activity Logging workers; Metrics Generation views; and the Administrative Dashboard

9.2 Matching Algorithm Design

The matching engine operates as a serialized Bull queue job to prevent race conditions (REQ-09). When a surplus is published, the algorithm executes four steps:

1. Query all registered recipients within 3 km via PostGIS ST_DWithin.
2. Score candidates using the composite formula:

$$\text{score} = 0.5 \text{ proximity} + 0.3 \text{ fairness_score} + 0.2 \text{ charity_priority}$$

where

$$\text{proximity} = 1 - \frac{\text{distance}}{3.0},$$

fairness_score encodes peripheral-user weighting, and charity_priority = 1.0 for verified CharityAgent instances.

3. Dispatch staggered push notifications by proximity tier: 500 m $\rightarrow$ 1 km $\rightarrow$ 3 km, with maximum 50 simultaneous dispatches per surplus to mitigate demand amplification (R9).
4. Apply automatic 15-minute timeout per tier with up to 3 reassignments per surplus, compressing the decision window by 50% at each reassignment to prevent expiry cascades (R8).

The algorithm guarantees that $\geq 90\%$ of offers ≥ 10 kg are assigned to verified charities first (REQ-03, REQ-11), directly addressing the equity-proximity tension identified as an emergent behavior in Workshop 1.

9.3 Technology Stack

Table 9.1: Technology Stack

Layer	Technology	Purpose
Frontend	React Native / Expo	Cross-platform Android & iOS from single codebase; GPS, camera, push notifications
Backend	NestJS (Node.js)	4 low-coupling modules: SurplusModule, MatchingModule, NotificationModule, AnalyticsModule
Database	PostgreSQL + PostGIS	ST_DWithin radius enforcement; spatial indexing for matching queries
Caching / Queue	Redis + Bull	Race-condition prevention via serialized jobs; JWT session caching
ORM	Prisma	Compile-time type safety; predictable migration workflows
Notifications	Firebase Cloud Messaging	Push delivery < 5 s to $\geq 95\%$ of active users (REQ-12)
Infrastructure	Docker Compose	One-command reproducible local deployment for academic prototype
State Management	Zustand + React Query	Lightweight UI state; server-state caching on the client
Analytics	Custom PostgreSQL views	KPI dashboards; supplier engagement early-warning alerts

9.4 Design Requirements

Seventeen measurable design requirements were derived directly from Workshop 1 findings. Selected requirements with their Workshop 1 traceability:

- **REQ-01:** 3 km radius enforcement (100% compliance, ± 50 m tolerance) — traces to spatial boundary sensitivity.
- **REQ-03:** Charity priority $\geq 90\%$ for offers ≤ 10 kg — traces to inequitable distribution emergent behavior.

- **REQ-06:** Pickup completion rate $\geq 85\%$ — traces to no-show cascade chaotic dynamic.
- **REQ-08:** Automatic reassignment (max 3 attempts) — traces to nonlinear cascade sensitivity.
- **REQ-09:** Race condition prevention via serialized queue — traces to high-demand event sensitivity.
- **REQ-12:** Notification delivery < 5 s for 95% of users — traces to demand amplification dynamic.
- **REQ-13:** Matching latency < 2 s — traces to peak concurrency performance requirement.
- **REQ-17:** Reliability scoring with 48-hour restriction after 3 no-shows in 7 days — traces to no-show behavioral risk.

Chapter 10

Risk Management (Workshop 3)

The risk management plan follows PMBOK and ISO 31000 standards, ensuring systematic identification, analysis, response planning, and continuous monitoring. All 10 risks identified in Workshop 1 were addressed through three-layer control strategies: proactive architectural mitigation, reactive contingency plans, and continuous KPI monitoring.

Table 10.1: Risk Register

ID	Risk	Mitigation	Contingency	Impact	KPI
R1	Race conditions (high-demand)	Serialized priority queue + DB locking	Fair lottery allocation	High	contention_rate
R2	High concurrency degradation	Redis caching + rate limiting	Graceful degradation	High	response_time
R3	Geolocation service failure	Redundant providers + client validation	Manual fallback (5 s)	High	geo_failures/hr
R4	No-show after assignment	Reliability scoring + 15 min timeout	Auto-reassignment (max 3×)	High	no_show_rate
R5	Informal bypass (WhatsApp)	Anomaly detection + audit log	Ranking penalties	Medium	anomaly_rate
R6	Supplier disengagement	Engagement metrics + feedback loops	Push notifications	Medium	supplier_trend
R7	Inequitable distribution	Fairness-weighted matching (30%)	Dynamic weight adjustment	High	equity_index
R8	No-show cascade failures	Reassignment cap (max 3)	Escalate to manual	High	chain_length
R9	Demand amplification	Staggered tiered notifications	Batch-size limiting	High	notifi_spike
R10	Negative feedback loop	Contextualized analytics dashboard	Personalized reports	Medium	publicate_rate

The identified risks form a dynamic system with interdependent feedback loops: no-show cascades (R4 → R8), notification-driven demand amplification (R9 → R1), and behavioral participation cycles (R5 → R6 → R10). The closed-loop control model — integrating all KPIs into the Analytics Layer with automated threshold-based alerts — transforms risk management from a reactive process into a dynamic, self-regulating system aligned with ISO 31000

principles.

10.1 Quality Assurance Framework

The quality assurance framework is structured in four sequential layers:

- **Unit Testing:** Each NestJS module is tested in isolation. Critical tests cover PostGIS `ST_DWithin` radius enforcement, composite scoring function accuracy, and Bull queue serialization under concurrent requests. Target: $\geq 80\%$ service function coverage.
- **Integration Testing:** End-to-end flows are validated, including donor publishes $\rightarrow$ charity assigned within 2 s; no-show triggers reassignment; geolocation failure activates fallback within 5 s; and staggered notifications respect tier order.
- **Acceptance Testing:** Load tests simulate ≥ 200 concurrent users during peak meal-hour scenarios. Acceptance is granted only when all minimum thresholds are simultaneously met over 48-hour continuous runs.
- **Pilot Validation:** A controlled 8-week pilot within the 3 km radius collects real KPI data, including kilograms redistributed, pickup completion rates, notification response times, and equity index values.

Chapter 11

Simulation and Validation (Workshop 4)

System behavior was validated through two complementary simulation paradigms executed across five experimental scenarios with 30 independent replications each ($n = 300$ total runs). The DES answers “how well does the infrastructure process behaviors?”, while the ABM answers “why and how do individual actors behave?”. The two approaches are deliberately complementary, and their convergent conclusions strengthen the validity of the findings.

11.1 Discrete-Event Simulation (DES)

The DES was implemented in Python SimPy 4.1 using a pipeline-process paradigm. Each surplus item traverses defined states:

POSTED $\rightarrow$ IN_QUEUE $\rightarrow$ ASSIGNED $\rightarrow$ PICKED_UP

or

NO_SHOW $\rightarrow$ (retry up to $3\times$) $\rightarrow$ EXPIRED.

The matching engine is modeled as a SimPy single-capacity Resource enforcing request serialization. Simulation time spans 15 continuous hours per replication (7:00–22:00). Surplus arrivals follow a Poisson process, while collection deadlines are drawn from $U[60, 180]$ minutes post-publication.

Five scenarios were designed to stress distinct system dimensions:

- **S01 Baseline:** Medium adoption, full mitigations.
- **S02 Early Stage:** Low adoption, partial mitigations.
- **S03 Stress Test:** High adoption, favorable behavior.
- **S04 Risk Test:** Elevated no-show (25–45%), staggered notifications disabled.
- **S05 Sensitivity:** Extreme no-show (35–55%), full mitigations.

Table 11.1: DES Simulation Results by Scenario (Mean $\pm$ 95% CI, $n = 30$)

Scenario	Recovery Rate	Completion Rate	No-Show Rate	Reassign Chain	CO ₂ (kg)
Baseline	0.986 $\pm$ 0.005	0.894 $\pm$ 0.012	0.095 $\pm$ 0.011	1.104 $\pm$ 0.014	2520 $\pm$ 106
Early Stage	0.983 $\pm$ 0.007	0.852 $\pm$ 0.020	0.138 $\pm$ 0.021	1.161 $\pm$ 0.028	1284 $\pm$ 87
Stress Test	0.989 $\pm$ 0.004	0.922 $\pm$ 0.007	0.069 $\pm$ 0.006	1.074 $\pm$ 0.007	5204 $\pm$ 150
Risk Test	0.968 $\pm$ 0.005	0.785 $\pm$ 0.015	0.202 $\pm$ 0.015	1.248 $\pm$ 0.023	2526 $\pm$ 131
Sensitivity	0.957 $\pm$ 0.009	0.703 $\pm$ 0.018	0.289 $\pm$ 0.016	1.393 $\pm$ 0.030	2478 $\pm$ 115

11.2 Agent-Based Model (ABM)

The ABM was implemented in Python Mesa 2.3.0. Five agent types represent the real system actors:

- DonorAgent (cafeterias/restaurants)
- BeneficiaryAgent (students/community)
- CharityAgent (verified organizations)
- VolunteerAgent (logistical support)
- SurplusAgent (food lots with defined lifecycle)

Each run simulates 864 ticks (6 simulated days, 10-minute steps). Agent locations follow a Gaussian spatial distribution ($\sigma = 1.0$ km, bounded to 3 km), concentrating approximately 68% of agents within 1 km of the Faculty, reflecting realistic campus density.

The composite scoring formula is:

$$\text{score} = 0.4 \times \text{proximity} + 0.4 \times \text{charity_bonus} + 0.2 \times \text{reliability}$$

where

$$\text{reliability} = \frac{\text{agent.reliability_score}}{100}.$$

Reliability scores update asymmetrically: +1 for successful pickup and -2 for no-show, bounded to $[0, 100]$. This asymmetric penalty creates a natural incentive gradient discouraging repeated unreliable behavior.

Table 11.2: ABM Simulation Results by Scenario (Mean $\pm$ 95% CI, $n = 30$)

Scenario	Recovery Rate	Completion Rate	Charity Share	No-Show Rate	Reassignments
Low Adoption	0.997 $\pm$ 0.004	0.804 $\pm$ 0.028	0.247 $\pm$ 0.050	0.051 $\pm$ 0.011	7.6 $\pm$ 1.4
Medium Adoption	0.999 $\pm$ 0.001	0.824 $\pm$ 0.015	0.249 $\pm$ 0.049	0.045 $\pm$ 0.005	13.0 $\pm$ 1.3
High Adoption	0.998 $\pm$ 0.002	0.819 $\pm$ 0.013	0.337 $\pm$ 0.034	0.054 $\pm$ 0.006	26.5 $\pm$ 2.3
No Mitigations	0.982 $\pm$ 0.007	0.638 $\pm$ 0.015	0.289 $\pm$ 0.045	0.108 $\pm$ 0.008	32.7 $\pm$ 2.0
Sensitivity	0.962 $\pm$ 0.009	0.558 $\pm$ 0.016	0.308 $\pm$ 0.046	0.151 $\pm$ 0.014	44.0 $\pm$ 2.7

11.3 Validation Results

Both simulation paradigms converge on three overarching conclusions:

- **Surplus Recovery Rate $\geq 80\%$.** Met in all scenarios across both paradigms. DES achieved 95.7–98.9%, while ABM achieved 96.2–99.9%. The automatic reassignment mechanism (REQ-08) prevents permanent surplus expiration even when individual no-show rates reach 35–55%.
- **Pickup Completion Rate $\geq 85\%$.** Partially met. DES satisfied the threshold in S01, S02, and S03 (89.4%, 85.2%, and 92.2%, respectively), but not in S04 (78.5%) or S05 (70.3%). ABM reached a maximum of 82.4% (Medium Adoption), constituting a near miss.
- **Charity Assignment Share $\geq 20\%$.** Met in all scenarios. DES consistently exceeded 97%, whereas ABM ranged between 24.7% and 33.7%. Both approaches confirm REQ-03, REQ-13, and the social fairness objective.

The infrastructure is confirmed as a non-bottleneck: DES engine utilization remains below 0.4% across all scenarios, matching latency remains approximately 1 second, and the system exhibits sufficient headroom for 10–20 $\times$ adoption growth without architectural changes.

11.4 System-Level Interpretation

The simulation results should be interpreted as evidence about the sociotechnical system rather than solely as platform performance metrics.

1. No-show behavior is the dominant system risk.

The near-perfect negative linear relationship between effective no-show rate and completion rate ($r \approx -0.95$) establishes a clear operational design constraint. Achieving consistent compliance with the 85% completion target requires organizational interventions such as commitment confirmation, reputation-based access tiers, or institutional partnerships with reliable charities.

2. Staggered notifications provide measurable mitigation value.

Scenario S04 (staggered notifications disabled) consistently underperformed S01 (staggered notifications enabled), quantifying the benefit of information-governance mechanisms at approximately 3–5 percentage points of completion rate.

3. The charity-priority mechanism is robust and adoption-invariant.

The fairness threshold is met in all scenarios, including the No Mitigations scenario, demonstrating that the equity objective is architecturally embedded rather than incidentally achieved.

Chapter 12

Implementation and Results

12.1 Functional Prototype

The functional prototype implements the full four-layer architecture documented in Workshop 2. The mobile application (React Native + Expo) provides cross-platform Android and iOS deployment from a single codebase, with native access to GPS, camera functionality for photo-based collection confirmation, and push notifications. State management uses Zustand for UI state and React Query for server-state caching with client-side invalidation.

The backend (NestJS, Node.js) is organized into four modules mirroring the Workshop 1 four-subsystem decomposition:

- SurplusModule
- MatchingModule
- NotificationModule
- AnalyticsModule

PostgreSQL with PostGIS enforces the 3 km radius constraint directly in SQL through `ST_DWithin` queries. Redis provides Bull queue management for serialized matching jobs and JWT session caching. The Prisma ORM ensures compile-time type safety and predictable migration workflows. All components are orchestrated through Docker Compose to provide reproducible local deployment.

12.2 Key Performance Indicators

Table 12.1: KPI Targets and Minimum Acceptable Thresholds

KPI	Target	Min. Acceptable
Surplus Recovery Rate	$\geq 80\%$	65%
Average Matching Time	$< 2\text{ s}$	$< 5\text{ s}$
Pickup Completion Rate	$\geq 85\%$	70%
Notification Delivery	$< 5\text{ s}$ (95%)	$< 10\text{ s}$
System Response Time	$< 500\text{ ms}$	$< 1\text{ s}$
Concurrent User Capacity	≥ 200	100
System Uptime	$\geq 99\%$	97%
Charity Assignment Share	$\geq 20\%$	10%

12.3 Risk Mitigation in Implementation

Each of the ten critical risks was addressed through specific implementation mechanisms consistent with the architectural decisions established in Workshop 2:

- **R1 (Race Conditions):** Database-level serialization using `SELECT FOR UPDATE`, combined with a fair lottery allocation mode when more than five simultaneous requests compete for the same surplus item.
- **R3 (Geolocation Failure):** Automatic fallback to manual address input with server-side radius validation, activated within 5 seconds of geolocation service failure.
- **R4 (No-Show Events):** Reliability scores are initialized at 100 points and reduced by 20 points for each no-show. Users receiving three no-shows within a seven-day period are restricted from claiming surplus items for 48 hours (REQ-17).
- **R6 (Supplier Disengagement):** Automated early-warning alerts are triggered when unclaimed surplus rates exceed 20% for three consecutive days. Individualized donor impact dashboards provide continuous engagement feedback.
- **R9 (Demand Amplification):** Staggered notification dispatch limits delivery to a maximum of 50 simultaneous push notifications per surplus publication. This mechanism was validated through simulation scenario S04.

Chapter 13

Discussion

The design decisions made across the four workshops reflect a deliberate balance between operational feasibility, social equity, and behavioral robustness. The 3 km geographic constraint, while restrictive, directly enables same-day redistribution feasibility by ensuring that all pickup points remain within pedestrian or short public transport distance—a critical condition in Bogotá university context.

The priority-queue matching algorithm proved to be the central design decision. By guaranteeing first-access notifications to verified charitable organizations for offers ≥ 10 kg, and incorporating a 30% fairness weight to counteract proximity bias for peripheral users, the system addresses both equity and efficiency objectives simultaneously. The 15-minute cascading response window per priority tier balances urgency with fairness, preventing listings from expiring unclaimed while maintaining the priority structure.

The sensitivity analysis from Workshop 1 proved foundational: the most critical systemic risks are behavioral dynamics rather than technical failures. No-show cascades, demand amplification through simultaneous notifications, and informal coordination networks that bypass the algorithm were all confirmed by simulation as genuine threats. The architectural responses—serialized Bull queues, staggered proximity-tier notifications, and user reliability scoring—directly address these nonlinear dynamics rather than treating them as edge cases.

The simulation results also reveal an important limitation: the platform can optimize coordination but cannot alone guarantee behavioral compliance. The 85% pickup completion threshold requires maintaining system-level no-show rates below 7% (DES) or 3–4% (ABM). Since no-show behavior is driven by individual decisions external to the software, sustained compliance requires organizational interventions: explicit commitment confirmation before assignment, institutional partnerships with charities guaranteeing reliability, and reputation-based access tiers that create meaningful incentives for consistent participation. These are system-level solutions that the platform supports but cannot unilaterally enforce.

Supplier disengagement under low demand was identified as the most structurally dangerous emergent behavior, creating a self-reinforcing negative cycle on both the supply and demand sides. The early-warning alert system and individualized impact dashboards are designed to break this cycle before it becomes structural, reflecting a key lesson from Workshop 1: platform sustainability depends as much on supplier motivation management as on technical matching efficiency.

Chapter 14

Conclusions and Future Work

This technical report documents the complete design, simulation, and validation of a mobile platform for real-time food surplus redistribution within the university environment of Universidad Distrital Francisco José de Caldas. By synthesizing the findings of four workshops, the report demonstrates that the proposed solution addresses a well-documented coordination gap through a combination of strict geospatial enforcement, same-day operational rules, a fairness-weighted priority-queue matching algorithm, and behavioral risk mitigation mechanisms.

The computational validation ($n = 300$ simulation runs across two paradigms and five scenarios) confirms that the system design is sound under realistic operational conditions. The Surplus Recovery Rate consistently exceeds 80% across all scenarios, while the Charity Assignment Share consistently exceeds 20%. The Pickup Completion Rate constitutes a near miss under nominal conditions (ABM: 82.4%) and a partial pass under favorable conditions (DES: 89.4%), with the remaining gap attributable to behavioral parameters rather than architectural limitations.

The overarching finding is that optimal platform performance requires sociotechnical interventions beyond software alone. Explicit commitment confirmation, institutional partnerships with reliable charitable organizations, and reputation-based participation incentives are necessary to sustain high completion rates. The platform provides the coordination infrastructure; however, its long-term effectiveness depends on continuous engagement with the human system it is designed to support.

Future work will focus on the following areas:

1. Adaptive no-show probabilities in simulation models, replacing fixed probability draws with dynamic functions based on accumulated reliability scores.
2. Extended reassignment limits, evaluating four and five retry attempts to determine whether Scenario S04 can consistently exceed the 85% completion threshold.
3. Pilot deployment at Universidad Distrital Francisco José de Caldas to collect empirical no-show distributions and calibrate simulation parameters using real-world operational data.
4. Machine-learning-based predictive surplus forecasting to improve matching efficiency and resource planning.
5. Environmental impact assessment incorporating lifecycle analysis of avoided CO₂ emissions and broader sustainability indicators.

References

Alcaldía de Bogotá (2024), 'Yo contribuyo, no desperdicio', Secretaría de Desarrollo Económico.

EatCloud Colombia (2025), 'Plataforma de redistribución de alimentos'. Accessed: 2026-06-02.

URL: <https://eatcloud.com>

OLIO (2024), 'How olio works'. Accessed: 2026-06-02.

URL: <https://olioapp.com>

United Nations Environment Programme (2021), 'Food waste index report 2021'.

URL: <https://www.unep.org/resources/report/unep-food-waste-index-report-2021>

Universidad Distrital Francisco José de Caldas (2026a), 'Workshop 1 – food waste reduction mobile application: A rigorous system analysis'.

Universidad Distrital Francisco José de Caldas (2026b), 'Workshop 2 – system design document'.

Universidad Distrital Francisco José de Caldas (2026c), 'Workshop 3 – robust system design and project management'.

Universidad Distrital Francisco José de Caldas (2026d), 'Workshop 4 – system simulation and validation'.

Appendix A

Simulation Parameters

The following table summarizes the key parameters used in both simulation paradigms (Workshop 4).

Table A.1: Simulation Parameters (DES and ABM)

Parameter	Value	Justification
Maximum operational radius	3.0 km	System specification (REQ-01)
Spatial distribution	Gaussian, $\sigma = 1.0$ km	Campus density modeling
ABM tick resolution	10 minutes	Operational window granularity
ABM ticks per simulation run	864 (6 simulated days)	Captures multi-day behavioral evolution
DES simulation horizon	900 min (15-hour day)	7:00–22:00 operating window
Publication window	24 ticks / 2–4 h before closing	Workshop 1 temporal boundary
Collection deadline	18 ticks / 1–3 h post-publication	Freshness constraint
Notification delay Ring 1→2	2 ticks / 10 min (DES)	Staged notification design
Notification delay Ring 2→3	4 ticks / 20 min (DES)	Staged notification design
Replications per scenario	30	Statistical confidence ($n = 30$)
Deterministic seeds	0–29 per scenario	Full reproducibility

Appendix B

Traceability Matrix (Requirements → Workshop 1 Findings)

Table B.1: Requirements Traceability Matrix

REQ	Requirement	Workshop 1 Finding	Simulation Result
REQ-01	3 km radius enforcement (100%, ± 50 m)	Spatial boundary constraint	100% in all scenarios
REQ-03	Charity priority $\geq 90\%$ for offers ≥ 10 kg	Inequitable distribution emergent behavior	DES: 97–99%; ABM: 25–34%
REQ-06	Pickup completion rate $\geq 85\%$	No-show cascade chaotic dynamic	Met in S01–S03 (DES)
REQ-07	Reliability scoring mechanism	No-show behavioral risk	No-show reduced to 4.5–5.4% (ABM)
REQ-08	Auto-reassignment (max 3 attempts)	Nonlinear cascade sensitivity	Chain length 1.07–1.39 (DES)
REQ-09	Race-condition prevention via serialized queue	High-demand surplus event sensitivity	Engine utilization $< 0.4\%$
REQ-12	Notification delivery < 5 s (95%)	Demand amplification chaotic dynamic	Structurally validated
REQ-13	Matching latency < 2 s	Peak concurrency performance	Uniformly ~ 1 s (DES)
REQ-17	Reliability restriction after 3 no-shows/7 days	Repeated no-show behavioral risk	Integrated in ABM reliability scoring